

Canevas standard — EDA & préparation avant modélisation de scoring

Principe directeur : ce canevas se suit dans l'ordre, sans sauter d'étape. Chaque étape a une règle de décision claire — pas de jugement au cas par cas.

0. Avant de toucher au code — cadrage (5 minutes, mais jamais sauté)

Question	Pourquoi ça détermine tout le reste
Qui utilise le score et pour quelle décision ?	Détermine si l'interprétabilité est obligatoire (→ élimine certains algos d'entrée de jeu)
Coût d'un faux positif vs faux négatif ?	Détermine la métrique cible (F1 pondéré, recall, precision) et plus tard le seuil
Le score sert à classer/prioriser, ou de vraie probabilité (%) ?	Détermine si la calibration sera nécessaire en fin de pipeline
Combien de lignes, combien de colonnes ?	Détermine si tu peux te permettre du visuel colonne par colonne ou si tu dois automatiser (>50 colonnes → automatiser, voir étape 4)

1. Chargement et premier coup d'œil

```
python

df.shape
df.info()
df.head()
df.sample(10)          # head() peut cacher des problèmes qui apparaissent aill
```

Règle : toujours `sample(10)` en plus de `head()`. Toujours vérifier `dtypes` — une colonne numérique lue comme `object` est un signal d'alerte immédiat (valeur texte cachée dans la colonne, séparateur mal interprété).

2. Qualité des données — règles de décision

2.1 Valeurs manquantes

```
python

df.isnull().sum()
df.isnull().mean() * 100
```

% manquant	Action par défaut
< 5 %	Imputation simple (médiane pour numérique, mode/"Unknown" pour catégoriel)
5-30 %	Imputation + créer une colonne binaire (<code>col_était_manquante</code>) (le fait d'être manquant est parfois lui-même informatif)
> 30 %	Envisager la suppression de la colonne, sauf si le manque est structurel et informatif (à vérifier avec <code>missingno</code>)
Corrélé à la target	⚠ signal de fuite de données potentielle — investiguer avant tout traitement automatique

python

```
import missingno as msno
msno.matrix(df)      # pattern aléatoire ou structuré ?
```

2.2 Doublons

python

```
df.duplicated().sum()
```

Règle : supprimer sauf justification métier explicite (ex : plusieurs transactions légitimes identiques).

2.3 Cohérence

Règle : vérifier systématiquement — valeurs négatives impossibles, dates hors plage, catégories mal orthographiées (`(df[col].unique())` sur les catégorielles).

3. Analyse univariée numérique — LE cœur de ce qu'on a travaillé ensemble

3.1 Diagnostic automatique (à faire pour TOUTES les colonnes numériques d'un coup — jamais colonne par colonne à l'œil sur un gros dataset)

python

```

import pandas as pd
from scipy import stats

def diagnostic_distributions(df, cols=None):
    if cols is None:
        cols = df.select_dtypes(include='number').columns
    resultats = []
    for col in cols:
        serie = df[col].dropna()
        if len(serie) < 3:
            continue
        skew = serie.skew()
        kurt = serie.kurtosis()
        if len(serie) < 5000:
            _, p_norm = stats.shapiro(serie.sample(min(len(serie), 5000), random_state=42))
        else:
            _, p_norm = stats.kstest(serie, 'norm', args=(serie.mean(), serie.std()))
        q1, q3 = serie.quantile([0.25, 0.75])
        iqr = q3 - q1
        n_outliers = ((serie < q1 - 1.5*iqr) | (serie > q3 + 1.5*iqr)).sum()
        resultats.append({
            'colonne': col,
            'skew': round(skew, 2),
            'kurtosis': round(kurt, 2),
            'p_normalite': round(p_norm, 4),
            'normale': p_norm > 0.05,
            'pct_outliers': round(n_outliers / len(serie) * 100, 1),
            'pct_manquant': round(df[col].isnull().mean() * 100, 1),
        })
    return pd.DataFrame(resultats).sort_values('skew', key=abs, ascending=False)

diag = diagnostic_distributions(df)

```

3.2 Règles de décision à partir du diagnostic

Constat	Seuil indicatif	Action
<code>abs(skew) > 1</code>	Asymétrie marquée	Candidate à transformation (log si positive et positive-only, Yeo-Johnson sinon) — mais seulement pour les modèles sensibles à l'échelle, voir section 5
<code>pct_outliers > 5 %</code>	Beaucoup d'outliers	Investiguer : erreur de saisie (corriger/supprimer) ou vraie valeur extrême (winsoriser ou transformer)
<code>kurtosis</code> très élevée	Queues épaisses	Vérifier si ce sont des erreurs ou un comportement réel (ex : fraude, événements rares)
<code>p_normalite < 0.05</code>	Non gaussien	Informatif seulement si tu utilises un modèle sensible à la forme (régression logistique, SVM) — sans effet pour les arbres
<code>std ≈ 0</code>	Variable quasi constante	Retirer — aucun pouvoir prédictif

3.3 Visualiser seulement les colonnes qui ressortent du diagnostic (pas toutes)

```
python
suspectes = diag[(diag['pct_outliers'] > 5) | (diag['skew'].abs() > 1)]['colonnes suspectes']

fig, axes = plt.subplots(len(suspectes[:12])//3 + 1, 3, figsize=(15, 4*(len(suspectes[:12])//3 + 1)))
for ax, col in zip(axes.flat, suspectes[:12]):
    sns.histplot(df[col], kde=True, ax=ax)
    ax.set_title(f"{col} (skew={df[col].skew():.2f})")
plt.tight_layout()
```

Si le dataset a beaucoup de colonnes (au-delà d'une cinquantaine), remplacer 3.1-3.3 par un rapport automatique : `ydata_profiling.ProfileReport(df, minimal=True)` — il signale lui-même les colonnes à problème.

4. Variables catégorielles

```
python
df[col].value_counts(normalize=True) * 100
```

Règle : catégories représentant < 1-2 % des lignes → regrouper en "Autre". Sinon bruit dans l'encodage sans signal.

5. LA SECTION CLÉ — préparation différenciée selon la famille d'algorithme

Principe : on ne prépare pas les données une seule fois pour tous les modèles. On

construit deux pipelines parallèles.

Traitement	Modèles à base d'arbres (Random Forest, XGBoost, LightGBM)	Modèles sensibles à l'échelle (régression logistique, SVM, KNN, réseaux de neurones)
Valeurs manquantes	Imputation simple (médiane/ mode) — ou nativement géré (XGBoost/LightGBM)	Imputation simple obligatoire
Transformation log/Box- Cox/Yeo-Johnson (asymétrie)	Inutile — invariant aux transformations monotones	Nécessaire si skew élevé (règle 3.2)
Scaling (StandardScaler/ RobustScaler)	Inutile	Obligatoire
Outliers	Peu sensible — traiter seulement si erreur de saisie avérée	Sensible — winsoriser ou transformer si vraie valeur extrême
Encodage catégoriel	One-hot ou natif (LightGBM/ CatBoost gèrent les catégories directement)	One-hot obligatoire (pas d'ordinal arbitraire)
Multicolinéarité (VIF)	Sans impact	À vérifier — $VIF > 5-10 \rightarrow$ retirer/ combiner une variable

5.1 Pipeline "arbres" — préparation minimale

```
python

from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier

pipeline_rf = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('model', RandomForestClassifier(random_state=42))
])
```

5.2 Pipeline "linéaire" — préparation complète

```
python
```

```

from sklearn.preprocessing import StandardScaler, PowerTransformer
from sklearn.linear_model import LogisticRegression

pipeline_logreg = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('power_transform', PowerTransformer(method='yeo-johnson')),
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(max_iter=1000))
])

```

Règle : toujours passer par un `Pipeline`, jamais transformer `(X)` à la main avant le split — c'est la garantie contre la fuite de données (le imputer/scaler apprend ses paramètres uniquement sur le train à chaque fold).

6. Analyse bivariable

Type	Outil	Test de confirmation
Numérique vs numérique	<code>df.corr()</code> + heatmap	VIF si corrélation entre features (pas avec la target) > 0.7
Catégorielle vs numérique	Boxplot par groupe	ANOVA (ou Kruskal-Wallis si non-normal)
Catégorielle vs catégorielle	Tableau croisé	Chi ² + V de Cramér

7. Variable cible

```

python

df['target'].value_counts(normalize=True)

```

Déséquilibre	Action
< 60/40	Rien de spécial
60/40 à 90/10	<code>class_weight='balanced'</code> , ou métrique robuste (F1, PR-AUC plutôt qu'accuracy)
> 90/10	+ envisager SMOTE/oversampling, documenter la stratégie ICI, pas à la modélisation

8. Fuite de données — checklist systématique avant de passer au modèle

- ☐ Aucune variable ne contient une information connue seulement *après* l'événement à prédire
 - ☐ Split train/test fait **avant** tout calcul de statistique (moyenne, imputation, scaling) — garanti si tout passe par un `Pipeline` + `cross_val_score`
 - ☐ Si données groupées (plusieurs lignes par entreprise/client) : même groupe jamais à cheval entre train et test (`GroupKFold` si nécessaire)
-

9. Split et validation — rappel de la méthode

python

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    stratify=y, random_state=42)

# Comparaison des candidats via k-fold, sur X_train uniquement
from sklearn.model_selection import cross_val_score
score_rf = cross_val_score(pipeline_rf, X_train, y_train, cv=5, scoring='roc_auc')
score_logreg = cross_val_score(pipeline_logreg, X_train, y_train, cv=5, scoring='roc_auc')
```

Règle : `stratify=y` systématique si classification, pour préserver les proportions de classes dans train et test.

10. Après le choix du modèle — ce qui reste à faire (hors EDA, rappel)

1. Tuning des hyperparamètres (`GridSearchCV`/`RandomizedSearchCV`) sur le(s) modèle(s) retenu(s) seulement
 2. **Calibration** du score si utilisé comme vraie probabilité (`CalibratedClassifierCV`) — pas nécessaire si le score sert juste à classer/prioriser
 3. Choix du seuil de décision selon le coût métier FP/FN (balayage de seuils + matrice de confusion à chacun — jamais 0,5 par défaut sans justification)
 4. Explicabilité (SHAP) si le modèle retenu est une boîte noire et que le contexte l'exige
-

Checklist finale — à cocher avant de passer à la modélisation

- ☐ Valeurs manquantes traitées, pattern vérifié (aléatoire ou pas)
- ☐ Doublons traités
- ☐ Diagnostic skew/kurtosis/outliers/normalité fait sur toutes les colonnes numériques
- ☐ Colonnes quasi constantes retirées
- ☐ Catégories rares regroupées
- ☐ Deux pipelines construits (arbres / linéaire), pas un seul traitement universel
- ☐ Multicolinéarité vérifiée (VIF) si modèle linéaire envisagé
- ☐ Target analysée, stratégie de déséquilibre décidée si nécessaire
- ☐ Fuite de données vérifiée (variables, split, groupes)

Les 12 questions à se poser avant chaque nouveau projet

1. Le dataset est-il complet ?
2. Y a-t-il des doublons ?
3. Les types de données sont-ils corrects ?
4. Y a-t-il des valeurs impossibles ?
5. La distribution est-elle normale ou asymétrique ? (et est-ce que ça compte pour l'algo visé ?)
6. Y a-t-il des outliers importants ?
7. Quelles variables sont les plus dispersées ?
8. Quelles variables sont fortement corrélées entre elles (multicolinéarité) ?
9. La variable cible est-elle équilibrée ?
10. Y a-t-il un risque de fuite de données ?
11. Le split a-t-il été fait avant tout calcul statistique ?
12. Qui utilise ce modèle, et quel est le coût relatif d'un faux positif vs faux négatif ?